

Buiten de controller: CREATE –UPDATE – DELETE (geen relatie)

Toevoegen (create)

ervan uitgaan dat de create method in de controller klaar is, maak je een button in t overzicht scherm genaamd “toevoegen” die het nieuwe toevoeg scherm opent.

```
private void btTrainerToevoegen_Click(object sender, EventArgs e)
{
    FrmTrainerToevoegen frmTrainerToevoegen = new FrmTrainerToevoegen();
    frmTrainerToevoegen.ShowDialog();
    FillListView();
}
```

Vervolgens maak je in het toevoeg scherm de benodigde textboxes en opslaan button, dan daarna klik je dubbel op de opslaan button.

```
private void btOpslaan_Click(object sender, EventArgs e)
{
    // stap A - controleren of alle velden ingevuld zijn
    if (tbTrainerNaam.Text.Length <= 0 ||
        tbTrainerEmail.Text.Length <= 0 ||
        tbTrainerTelefoonnummer.Text.Length <= 0 ||
        tbSpecialisatie.Text.Length <= 0)
    {
        MessageBox.Show("Vul alle velden in.");
        return;
    }

    // stap B - nieuw TrainerModel object maken
    TrainerModel trainer = new TrainerModel();

    // stap C - ingevulde gegevens in het model zetten
    trainer.TrainerNaam = tbTrainerNaam.Text;
    trainer.TrainerEmailAdres = tbTrainerEmail.Text;
    trainer.TrainerTelefoonnummer = tbTrainerTelefoonnummer.Text;
    trainer.Specialisatie = tbSpecialisatie.Text;

    // stap D - controller object maken
    TrainerController controller = new TrainerController();

    // stap E - Create method aanroepen
    int result = controller.Create(trainer);

    // stap F - controleren of opslaan gelukt is
    if (result > 0)
    {
        MessageBox.Show("Trainer succesvol toegevoegd.");
        this.Close();
    }
    else
    {
        MessageBox.Show("Trainer is niet toegevoegd.");
    }
}
```

Wijzigen (UPDATE)

Ervan uitgaand dat de UPDATE method in de controller klaar is, maak je in het overzicht scherm een button genaamd “wijzigen” die het nieuwe wijzig scherm opent.

```
private void btWijzigen_Click(object sender, EventArgs e)
{
    // stap A - controleer of er precies 1 trainer geselecteerd is
    if (lvTrainer.SelectedItems.Count != 1)
    {
        MessageBox.Show("Selecteer 1 trainer om te wijzigen.");
        return;
    }

    // stap B - haal het gekoppelde model op uit de Tag
    TrainerModel trainer = (TrainerModel)lvTrainer.SelectedItems[0].Tag;

    // stap C - maak het wijzig-form aan en geef de trainer mee
    FrmTrainerWijzigen frmTrainerWijzigen = new FrmTrainerWijzigen(trainer);

    // stap D - open het wijzig-form
    frmTrainerWijzigen.ShowDialog();

    // stap E - ververs het overzicht
    FillListView();
}
```

Hierna maak je in de code van het wijzig-form, bovenaan in de partial class, een variabele aan om de geselecteerde trainer op te slaan.

```
private TrainerModel trainerItem;
```

Daarna maak je een constructor aan recht onder InitializeComponent(TrainerModel trainer);.

```
trainerItem = trainer;
```

Daarna vul je in het Load-event de velden met de oude gegevens van de geselecteerde trainer. Hierdoor ziet de gebruiker meteen wat er nu in de database staat.

```
private void FrmTrainerWijzigen_Load(object sender, EventArgs e)
{
    tbTrainerNaam.Text = trainerItem.TrainerNaam;
    tbTrainerEmail.Text = trainerItem.TrainerEmailAdres;
    tbTrainerTelefoonnummer.Text = trainerItem.TrainerTelefoonnummer;
    tbSpecialisatie.Text = trainerItem.Specialisatie;
}
```

Als laatste maak je de opslaan-functionaliiteit. Hierbij lees je de aangepaste waarden uit de velden, zet je die terug in het bestaande trainerItem object en roep je de Update-method aan.

```
private void btWijzigingOpslaan_Click(object sender, EventArgs e)
{
    // stap A - controller object maken
    TrainerController controller = new TrainerController();

    // stap B - controleren of alle velden ingevuld zijn
    if (string.IsNullOrEmpty(tbTrainerNaam.Text) ||
        string.IsNullOrEmpty(tbTrainerEmail.Text) ||
        string.IsNullOrEmpty(tbTrainerTelefoonnummer.Text) ||
        string.IsNullOrEmpty(tbSpecialisatie.Text))
    {
        MessageBox.Show("Vul alle velden in.");
        return;
    }

    // stap C - nieuwe waarden in het model zetten
    trainerItem.TrainerNaam = tbTrainerNaam.Text;
    trainerItem.TrainerEmailAdres = tbTrainerEmail.Text;
    trainerItem.TrainerTelefoonnummer = tbTrainerTelefoonnummer.Text;
    trainerItem.Specialisatie = tbSpecialisatie.Text;

    // stap D - Update method aanroepen
    int result = controller.Update(trainerItem);

    // stap E - controleren of wijzigen gelukt is
    if (result > 0)
    {
        MessageBox.Show("Trainer succesvol gewijzigd.");
        this.Close();
    }
    else
    {
        MessageBox.Show("Trainer is niet gewijzigd.");
    }
}
```

Verwijderen (DELETE)

ervan uitgaan dat de DELETE method in de controller klaar is, maak je in het overzicht scherm een button genaamd “verwijderen” vervolgens klik je dubbel op die knop.

```
private void btVerwijderen_Click(object sender, EventArgs e)
{
    // stap A - controleer of er precies 1 item geselecteerd is
    if (lvTrainer.SelectedItems.Count != 1)
    {
        MessageBox.Show("Selecteer 1 trainer om te verwijderen.");
        return;
    }

    try
    {
        // stap B - maak controller object aan
        TrainerController controller = new TrainerController();

        // stap C - haal het gekoppelde model uit de Tag
        TrainerModel trainer = (TrainerModel)lvTrainer.SelectedItems[0].Tag;

        // stap D - roep de Delete method aan
        controller.Delete(trainer);

        // stap E - ververs het overzicht
        FillListView();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Er is een fout opgetreden: " + ex.Message);
    }
}
```